

Báo Cáo Thuật Toán: Aura Tracker (Hệ Thống Giám Sát Aura)

Nhóm 13 (Update 2)

Ngày 20 tháng 11 năm 2025

Tóm tắt nội dung

Báo cáo trình bày giải pháp cho bài toán Aura Tracker qua 3 giai đoạn (Sub-tasks), phân tích sâu vào các chiến lược thiết kế thuật toán từ Vết cạm đến Biến đổi và Trị.

1 Sub-task 1: Kích thước dữ liệu nhỏ ($N, M \leq 1000$)

1.1 Phương pháp thiết kế thuật toán (0.4đ)

- **Chiến lược thiết kế: Brute Force (Vết cạm).**
- **Phân tích:** Đây là phương pháp tiếp cận trực tiếp nhất, dựa trên định nghĩa của bài toán mà không qua bất kỳ bước biến đổi hay tối ưu hóa cấu trúc nào. Thuật toán duyệt qua tất cả các khả năng có thể (cặp Hunter - Vùng) để tìm ra kết quả đúng.
- **Mô tả cài đặt:** Với mỗi Hunter, ta dùng vòng lặp duyệt qua toàn bộ danh sách N hình chữ nhật. Kiểm tra điều kiện bao hàm ($x_{min} \leq x \leq x_{max}$) và ($y_{min} \leq y \leq y_{max}$). Trong các hình thỏa mãn, chọn hình có diện tích nhỏ nhất.

1.2 Lý do phương pháp phù hợp (0.3đ)

Với kích thước $N, M \leq 1000$, tổng số phép toán là 10^6 , rất nhỏ so với khả năng xử lý của máy tính (10^8 ops/sec). Chiến lược Vết cạm phù hợp vì cài đặt nhanh, dễ kiểm soát lỗi (bug-free), và dùng làm cơ sở (baseline) để kiểm tra tính đúng đắn của các thuật toán phức tạp hơn.

1.3 Phân tích độ phức tạp (0.3đ)

- **Thời gian:** $O(M \times N)$.
- **Không gian:** $O(N + M)$ (chỉ lưu mảng đầu vào).

2 Sub-task 2: Kích thước lớn nhưng Tọa độ nhỏ ($W, H \leq 1000$)

2.1 Phương pháp thiết kế thuật toán (0.4đ)

- **Chiến lược thiết kế: Space-Time Trade-off (Đánh đổi không gian lấy thời gian).**
- **Phân tích:** Cụ thể đây là kỹ thuật **Input Enhancement (Cải tiến dữ liệu đầu vào)**. Ta chấp nhận tốn thêm bộ nhớ để tiền xử lý dữ liệu (xây dựng bản đồ/ma trận), giúp việc truy vấn sau này diễn ra tức thì.
- **Mô tả cài đặt:** Tạo ma trận $Matrix[W][H]$. Sắp xếp các hình chữ nhật theo thứ tự giảm dần về kích thước (hoặc cấp độ bao hàm). Duyệt và tô màu ID hình chữ nhật lên ma trận. Khi truy vấn Hunter, chỉ cần tra cứu trực tiếp $Matrix[x][y]$.

2.2 Lý do phương pháp phù hợp (0.3đ)

Giới hạn tọa độ nhỏ (1000×1000) cho phép tạo mảng 2 chiều (1 triệu phần tử) vừa vặn trong RAM. Chiến lược đánh đổi bộ nhớ này giúp giảm độ phức tạp truy vấn từ $O(N)$ (vết cạm) xuống còn $O(1)$, giải quyết tốt vấn đề khi số lượng Hunter M rất lớn.

2.3 Phân tích độ phức tạp (0.3đ)

- **Thời gian:** Tiền xử lý $O(N \cdot W \cdot H)$ (hoặc $O(W \cdot H)$ tối ưu), Truy vấn $O(1)$.
- **Không gian:** $O(W \times H)$.

3 Sub-task 3: Ràng buộc đầy đủ ($N, M \leq 10^5, W, H \leq 10^9$)

3.1 Phương pháp thiết kế thuật toán (0.4đ)

Ở mức độ này, chúng tôi kết hợp hai chiến lược thiết kế nâng cao:

3.1.1 1. Transform and Conquer (Biến đổi và Trị)

- **Kỹ thuật: Sweep-line (Quét dòng) và Coordinate Compression (Nén tọa độ).**
- **Phân tích:**
 - *Biến đổi chiều (Dimensionality Reduction):* Kỹ thuật Quét dòng biến đổi bài toán hình học 2D tính thành chuỗi các bài toán cập nhật cấu trúc dữ liệu 1D động theo thời gian.
 - *Biến đổi miền giá trị:* Nén tọa độ biến đổi miền giá trị thừa thớt $[0, 10^9]$ thành miền giá trị đặc khít $[0, 2N]$, giúp áp dụng được các cấu trúc dữ liệu mảng.

3.1.2 2. Divide and Conquer (Chia để trị)

- **Kỹ thuật: Segment Tree (Cây phân đoạn).**
- **Phân tích:** Segment Tree áp dụng triệt để tư tưởng Chia để trị. Để quản lý một đoạn $[L, R]$, nó chia đôi thành $[L, Mid]$ và $[Mid + 1, R]$, giải quyết bài toán trên hai nửa rồi tổng hợp lại. Điều này cho phép thực hiện các thao tác cập nhật vùng (Range Update) và truy vấn điểm (Point Query) với chi phí Logarithmic.

3.2 Lý do phương pháp phù hợp (0.3đ)

- Tọa độ lớn (10^9) khiến chiến lược *Space-Time Trade-off* (Sub-task 2) thất bại → Cần **Transform and Conquer** (Nén tọa độ).
- Số lượng phần tử lớn (10^5) khiến chiến lược *Brute Force* (Sub-task 1) thất bại → Cần **Divide and Conquer** (Segment Tree) để đạt độ phức tạp $O(N \log N)$.

3.3 Phân tích độ phức tạp (0.3đ)

- **Thời gian:** $O((N + M) \log(N + M))$ do chi phí sắp xếp sự kiện và thao tác trên cây $O(\log N)$.
- **Không gian:** $O(N + M)$ để lưu trữ cây Segment Tree và mảng sự kiện.